

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

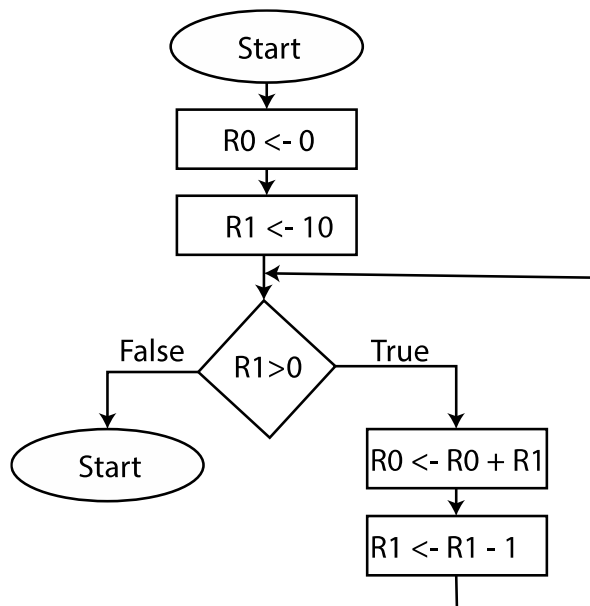
Note: Hvis du mangler specifikationer, kan du definere dine egne og fortsætte opgaveløsningen. Ikke alle begreber/beskrivelser er oversat til dansk. Eksempelvis er ordene "run-time stack" og "pointer" ikke oversat. I enkelte tilfælde er ikke oversatte betegnelser indrammet i anførselstegn ("...").

Opgave 1 (10%)

Givet nedestående LC3 maskinkodeprogram, som starter i adresse x3000:

x3000	0101000000100000	;	AND R0, R0, 0
x3001	0010001000000101	;	LD R1 val
x3002	0000110000000011	;loop	BRNZ done
x3003	0001000000000001	;	ADD R0 R0 R1
x3004	0001001001111111	;	ADD R1 R1 #-1
x3005	0000111111111100	;	BRNZP loop
x3006	1111000000100101	;done	TRAP x25
x3007	0000000000001010	;val	.FILL x000A

a) Tegn et flowdiagram for maskinkoden



b) Forklar med dine egne ord, hvad programmet laver

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

Programmet beregner i R0 summen af alle heltal fra 1 til det positive tal, vi læser fra adressen x3007. I vores tilfælde har vi værdien 10 på adressen x3007, og summen af heltal fra 1 til 10 er 55.

Opgave 2(30%)

Skriv et LC3 assembler program, som starter fra adressen x3000. Programmet læser en tekststreng, som vi kalder **inStr**, som starter fra adresse x4000. Strengen afsluttes med '\0', som har ASCII værdi 0. Programmet giver output i et array, som vi kalder **histogram**, som starter fra adresse x5000, og som indeholder 16 værdier.

Ved programmets start skal de 16 pladser i arrayet **histogram** initialiseres til værdien 0. Derpå skal programmet gennemgå hver karakter i **inStr**, bestemme de sidste 4 bit af ASCII-værdien og inkrementere værdien af pågældende plads i **histogram** arrayet. Eksempelvis har bogstav 'H' ASCII værdien x48. Værdien af de sidste 4 bit er 8, hvorfor positionen i histogram arrayet med indeks 8 skal inkrementeres med 1 ($\text{histogram}[8] = \text{histogram}[8] + 1$).

Indeholder **inStr** strengen "test", vil følgende værdier ligge i **histogram** arrayet efter endt udførelse:

- Værdien 1 på adresse x5003 ($\text{histogram}[3]$) svarende til en forekomst af 's',
- Værdien 2 på adressen x5004 ($\text{histogram}[4]$) svarende til to forekomster af 't',
- Værdien 1 på adresse x5005 ($\text{histogram}[5]$) svarende til en forekomst af 'e'
- Værdien 0 på de resterende adresser mellem x5000 og x500F.

a) Tegn et flowdiagram for programmet.

Skriftlig prøve/dato: 27 maj 2021

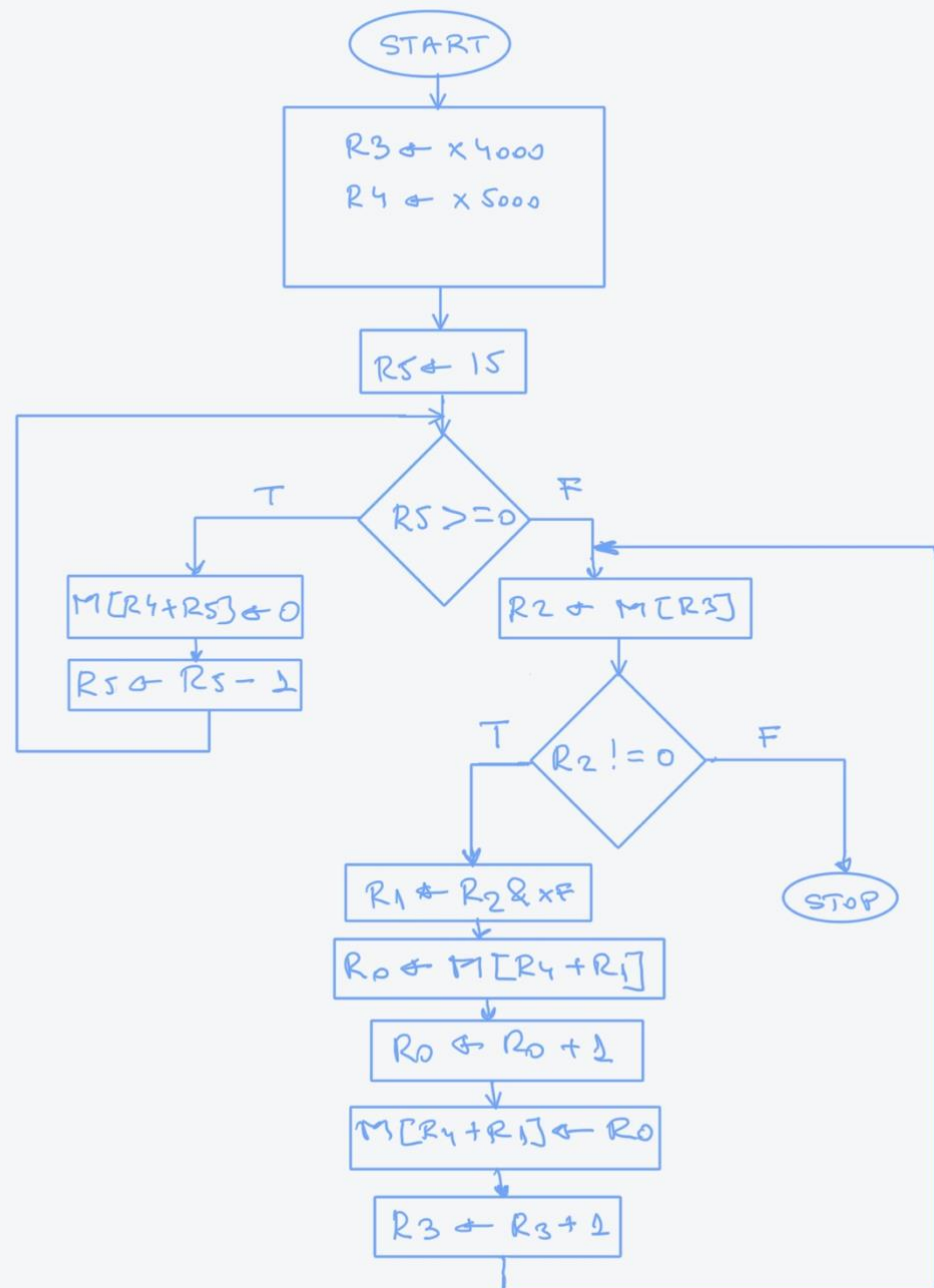
Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%



Vigtigere registre:

R2 – værdi af den nuværende bogstav R3 peger på

R3 – pointer til et bogstav i inStr

R4 – pointer til starten af histogram array

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

b) Skriv programmet i LC-3 assembler.

```
.orig x3000
    LD R3, inStr      ;pointer to the start of input string
    LD R4, histogram ;pointer to the start of histogram
                        ; array
    ;Initializing the histogram with zeroes
    AND R0, R0, #0    ; R0 <- 0
    LD R5, maxindex ;R5 <- 15 //index in the histogram array
loop1  BRn loop2
    ADD R6, R4, R5     ; The address of the element in the
                        ; histogram

    STR R0, R6, #0
    ADD R5, R5, #-1
    BRnzp loop1
loop2  LDR R2, R3, #0   ; Current letter in the string
    BRz done
    AND R1, R2, xF
    ADD R1, R1, R4
    LDR R0, R1, #0
    ADD R0, R0, #1
    STR R0, R1, #0
    ADD R3, R3, #1
    BRnzp loop2
done   HALT

inStr      .fill x4000
histogram  .fill x5000
maxindex   .fill #15
    .end
```

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

Opgave 3(20%)

Jakob ønsker at skrive et C program, hvori han initialiserer et array indeholdende 3 strenge, så programmet kan give følgende output på konsollen efter udførelse:

```
strings[0]=Hi  
strings[1]=Hello  
strings[2]=Hola
```

Jakob har skrevet efterfølgende program, der hverken er færdigt eller korrekt:

```
#include <stdio.h>  
  
int main() {  
    char** strings;  
    char string[]="Hola";  
    char* strArray[3];  
    strings[0][0]='H';  
    strings[0][1]='i';  
    strings[0][2]='\0';  
    strings[1]="Hello";  
  
    for (int i=0;i<3;i++){  
        printf("string[i]=%s",strings[i]);  
    }  
    return 0;  
}
```

- a) Hjælp Jakob ved at færdiggøre programmet, så det giver det ønskede output.
Du må ikke ændre, slette eller udkommentere de linjer af koden, Jakob allerede har skrevet. Men du må gerne tilføje linjer kode til programmet.

```
int main() {  
    //First 3 lines cannot be changed  
    char** strings;  
    char string[]="Hola";  
    char* strArray[3];  
    /***  
    char str0[3];          //L1  
    strings=strArray;      //L2  
    strArray[0]=str0;      //L3  
  
    //Following lines cannot be changed  
    strings[0][0]='H';  
    strings[0][1]='i';  
    strings[0][2]='\0';  
    strings[1]="Hello";  
    /***  
    strings[2]=string;      //L4  
    //Following lines cannot be changed
```

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

```
for (int i=0;i<3;i++){  
    printf("strings[%d]=%s\n",i,strings[i]);  
}  
return 0;  
//***  
}
```

- b) Det antages nu, at koden fra spørgsmål 3a udføres på en LC3 computer. Det antages, at stak-pointeren i R6 har værdien FFFF, når stakken er tom.

Angiv indholdet af hele "runtime" stakken umiddelbart før udførelsen af "return 0;" fra **main** funktionen.

Hvis du ikke har besvaret spørgsmål 3a, kan du bruge den kode, som Jakob har skrevet i besvarelse af dette spørgsmål.

FFF0	str0[0] = 'H'
FFF1	str0[1] = 'i'
FFF2	str0[2] = '\0'
FFF3	*strArray[0] = str[0] = xFFF0
FFF4	*strArray[1] = &"Hello"
FFF5	*strArray[2] = string = xFFF6
FFF6	string[0] = 'H'
FFF7	string[1] = 'o'
FFF8	string[2] = 'l'
FFF9	string[3] = 'a'
FFFA	string[4] = '\0'
FFFB	**strings = strArray = xFFF3
FFFC	Dynamic Link
FFFD	Return Address
FFFE	Return Value
FFFF	

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

Hvis vi bruger kun den kode som Jakob har skrevet så vil stakken være:

FFF3	*strArray[0] = ?
FFF4	*strArray[1] = ?
FFF5	*strArray[2] = ?
FFF6	string[0] = 'H'
FFF7	string[1] = 'o'
FFF8	string[2] = 'l'
FFF9	string[3] = 'a'
FFFA	string[4] = '\0'
FFFB	**strings = ?
FFFC	Dynamic Link
FFFD	Return Address
FFFE	Return Value
FFFF	

Opgave 4(40%)

Givet nedenstående C program, hvori implementeringen af enkelte funktioner mangler:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct my_node node;
struct my_node {
    int val;
    node* next;
};
node* header;

void initialize_list();
void show_list();
void append_node(int val);
void remove_duplicates(node* startNode);
void remove_all_duplicates();
void reverse_list();

int values[]={3,5,3,5,8,4,6,3,9,4};

int main() {
    setbuf(stdin,0);
    printf("Creating the list\n");
    initialize_list();
    show_list();
    printf("Removing all duplicates\n");
    remove_all_duplicates();
    show_list();
}
```

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

```
    printf("Reversing the list\n");
    reverse_list();
    show_list();
    printf("Reversing the list again\n");
    reverse_list();
    show_list();
    return 0;
}

//Adding a new node at the end of the list
void append_node(int val) {
    //Missing Code 1
}

void initialize_list() {
    int i;
    for (i=0;i<10;i++) {
        append_node(values[i]);
    }
}

void show_list() {
    node* currentNode=header;
    printf("Values contained in the list:\n");
    while(currentNode) {
        printf("%d\t",currentNode->val);
        currentNode=currentNode->next;
    }
    printf("\n");
}

//Look in the rest of the list for other nodes with the same val value as the
// startNode argument for this function and remove the duplicate nodes from
// the list starting at startNode
void remove_duplicates(node* startNode) {
    //Missing Code 2
}

//Remove all the duplicate nodes from the list given by the global variable
// header
void remove_all_duplicates() {
    //Missing Code 3
}

//Reversing the order of the nodes in the list given by the global variable
// header. The last node in the list becomes the first, the second last node
// becomes the second etc.
void reverse_list() {
    //Missing Code 4
}
```

Note:

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

- Den globale variable "header" er en pointer, som peger på den første node af den "single linked list", som vi bruger i alle funktioner i dette program.
- Du må ikke ændre den eksisterende kode. Du må kun tilføje C kode, der hvor der står **//Missing code**

Hvis programmet er færdigt, og alle funktioner er korrekt implementerede, forventes følgende output:

Creating the list

Values contained in the list:

3 5 3 5 8 4 6 3 9 4

Removing all duplicates

Values contained in the list:

3 5 8 4 6 9

Reversing the list

Values contained in the list:

9 6 4 8 5 3

Reversing the list again

Values contained in the list:

3 5 8 4 6 9

- Udfyld funktionen **append_node()** på det sted, hvor kommentaren "Missing code 1" står. Denne funktion laver en ny node indeholdende værdien val, og funktionen tilføjer den ny node til enden af listen.
- Udfyld funktionen **remove_duplicates(node* startNode)** på det sted, hvor kommentaren "Missing code 2" står. Denne funktion betragter alle noder i den linkede liste efter noden startNode og fjerner de noder fra listen, som har samme værdi val som startNode.
- Udfyld funktionen **remove_all_duplicates()** på det sted, hvor kommentaren "Missing code 3" står. Denne funktion fjerner de noder af en given værdi, der optræder mere end en gang, fra listen; Efter fjernelse indeholder listen netop en instans af alle værdier. Du kan kalde funktionen **remove_duplicates**, som du skrev under 4b, når du implementerer funktionen **remove_all_duplicates**.
- Udfyld funktionen **reverse_list ()** på det sted, hvor kommentaren "Missing code 4" står. Denne funktion spejlvender den linkede liste, som "header" peger på. Elementernes indbyrdes rækkefølge ændres, så det sidste element i listen bliver det første, det næstsidste bliver det andet osv. "header" vil ende med at pege på det sidste element i den oprindelige liste.

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

Hele kildekoden for programmet:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct my_node node;
struct my_node {
    int val;
    node* next;
};
node* header;

void initialize_list();
void show_list();
void append_node(int val);
void remove_duplicates(node* startNode);
void remove_all_duplicates();
void reverse_list();

int values[]={3,5,3,5,8,4,6,3,9,4};

int main() {
    setbuf(stdin,0);
    printf("Creating the list\n");
    initialize_list();
    show_list();
    printf("Removing all duplicates\n");
    remove_all_duplicates();
    show_list();
    printf("Reversing the list\n");
    reverse_list();
    show_list();
    printf("Reversing the list again\n");
    reverse_list();
    show_list();
    return 0;
}

//Adding a new node at the end of the list
void append_node(int val){
    node* currentNode=header;
    node* previousNode=NULL;
    node* newNode=(node*)malloc(sizeof(node));

    while(currentNode){
        previousNode=currentNode;
        currentNode=currentNode->next;
    }
    newNode->val=val;
    newNode->next=NULL;
    if (previousNode)
        previousNode->next=newNode;
    else
        header=newNode;
```

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

```
}

void initialize_list() {
    int i;
    for (i=0; i<10; i++) {
        append_node(values[i]);
    }
}

void show_list() {
    node* currentNode=header;
    printf("Values contained in the list:\n");
    while(currentNode) {
        printf("%d\t", currentNode->val);
        currentNode=currentNode->next;
    }
    printf("\n");
}

//Look in the rest of the list for other nodes with the same value like this
// one and remove the duplicates
void remove_duplicates(node* startNode) {
    node* currentNode;
    node* previousNode;
    if (startNode) {
        currentNode=startNode->next;
        previousNode=startNode;
    } else
        return;

    while(currentNode) {
        if(currentNode->val==startNode->val) { //If match remove current node
            previousNode->next=currentNode->next;
        } else
            previousNode=currentNode;
        currentNode=currentNode->next;
    }
}

//Remove all the duplicate nodes from the list given by the global variable
// header
void remove_all_duplicates() {
    node* currentNode=header;
    while(currentNode) {
        remove_duplicates(currentNode);
        currentNode=currentNode->next;
    }
}
```

Skriftlig prøve/dato: 27 maj 2021

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpe midler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–30%, Opgave 3–20%, Opgave 4–40%

```
//Reversing the order of the nodes in the list given by the global variable  
// header. The last node in the list becomes the first, the second last node  
// becomes the second etc.  
void reverse_list() {  
    node* current=header;  
    node* newHeader=NULL;  
    node* next;  
    while (current) {  
        next=current->next;  
        current->next=newHeader;  
        newHeader=current;  
        current = next;  
    }  
    header=newHeader;  
}
```